

ASSIGNMENT

Môn học: Cơ sở dữ liệu (DBI202)

Đề tài: Video Game Sales Database Management System

Lớp:	DBI202 M1 BL5
Giảng viên hướng dẫn:	QuanTL3

Nhóm 7 - Thành viên thực hiện

MSSV	Họ và tên
HE191422	Lê Minh Hoàng
HE141378	Bùi Hà Vi
HE170431	Phạm Thành Trung

Tháng 8, 2026

MỤC LỤC

MỤC LỤC.....	2
DANH MỤC HÌNH VẼ.....	5
DANH MỤC BẢNG BIỂU	5
1. Đặt vấn đề	6
2. Mô tả nghiệp vụ	6
3. Mô hình ER (Entity - Relationship).....	6
Giải thích các entity và quan hệ	7
Lý do thiết kế 2 bảng trung gian	8
4. Mô hình quan hệ, phụ thuộc hàm và chuẩn hoá 3NF	8
Vì sao các bảng đạt chuẩn 3NF.....	9
5. Đặc tả yêu cầu dữ liệu (Data Dictionary)	9
Bảng danh mục (genre, platform, publisher, region).....	9
Bảng game.....	10
Bảng game_publisher (bảng trung gian).....	11
Bảng game_platform (bảng trung gian).....	11
Bảng region_sales (PK phức hợp)	12
6. Danh sách ràng buộc dữ liệu.....	13
7. Cài đặt vật lý trên SQL Server.....	13
7.1. Tạo database và các bảng.....	13
7.2. Index phục vụ hiệu năng truy vấn.....	13
7.3. Nạp dữ liệu mẫu (insert.sql).....	14
Bảng platform (31 dòng).....	14
Bảng genre (12 dòng).....	14
Bảng publisher (577 dòng).....	14
Bảng region (4 dòng)	14
Bảng game (11.360 dòng).....	14
Bảng game_publisher (11.732 dòng).....	14
Bảng game_platform (16.326 dòng).....	14

Bảng region_sales (65.320 dòng)	14
8. Truy vấn (Queries)	14
8.1. Inner Join.....	15
8.2. Outer Join.....	15
8.3. Subquery trong WHERE.....	15
8.4. Subquery trong FROM	15
8.5. Group By + Aggregate.....	15
8.6. Xu hướng doanh số theo năm	15
8.7. Nền tảng bán chạy nhất.....	15
8.8. Game bán chạy nhất trên mỗi nền tảng.....	15
8.9. Doanh số thể loại theo khu vực.....	15
9. Transaction.....	15
10. Trigger.....	16
11. Procedure / Function	16
11.1. sp_GetTopGamesByRegion	16
11.2. sp_AddNewGame	16
11.3. sp_UpdateGameSales	16
11.4. sp_DeleteGame	16
11.5. sp_SearchGames	17
11.6. fn_GetTotalSalesByGame	17
12. Áp dụng AI.....	17
NL2SQL Agent - hỏi đáp dữ liệu bằng ngôn ngữ tự nhiên	17
13. Kết luận, hướng phát triển	18
Kết luận.....	18
Hạn chế	19
Hướng phát triển	19
14. Tỷ lệ đóng góp	19
PHỤ LỤC: CÂU LỆNH SQL VÀ KẾT QUẢ THỰC THI.....	20
Phụ lục 6. Ràng buộc dữ liệu	20

Phụ lục 7. Cài đặt vật lý trên SQL Server.....	21
Phụ lục 8. Truy vấn.....	23
Phụ lục 9-10. Transaction và Trigger	25
Phụ lục 11. Procedure / Function.....	27

DANH MỤC HÌNH VẼ

Hình 1. Sơ đồ ER hệ thống Group7 - Video Game Sales.....	7
Hình 2. Giao diện chat NL2SQL Agent.....	18
Hình 3. Trang /admin - giám sát request theo thời gian thực	18

DANH MỤC BẢNG BIỂU

Bảng 1. Các thực thể và quan hệ trong mô hình ER.....	7
Bảng 2. Lược đồ quan hệ và phụ thuộc hàm của 8 bảng	8
Bảng 3. Đặc tả các bảng danh mục (genre, platform, publisher, region)	9
Bảng 4. Đặc tả bảng game	10
Bảng 5. Đặc tả bảng game_publisher	11
Bảng 6. Đặc tả bảng game_platform.....	11
Bảng 7. Đặc tả bảng region_sales	12
Bảng 8. Số dòng dữ liệu nạp cho từng bảng	14
Bảng 9. Tỷ lệ đóng góp của các thành viên	19

1. Đặt vấn đề

Ngành công nghiệp game phát triển mạnh trên toàn cầu, với hàng chục nghìn tựa game được phát hành mỗi năm trên nhiều nền tảng (platform) khác nhau, bởi nhiều nhà phát hành (publisher), thuộc nhiều thể loại (genre), và ghi nhận doanh số ở nhiều khu vực địa lý (region) khác nhau. Việc quản lý và phân tích dữ liệu doanh số game một cách có hệ thống giúp nhà phát hành và nhà phân phối đưa ra quyết định kinh doanh: nên đầu tư phát triển thể loại game nào, phát hành trên nền tảng nào, ở khu vực nào có tiềm năng nhất.

Nhóm 7 lựa chọn xây dựng hệ cơ sở dữ liệu quan hệ Group7 quản lý dữ liệu bán game (dựa trên dataset Video Game Sales) trên Microsoft SQL Server, đồng thời phát triển thêm một chatbot nội bộ (NL2SQL Agent) hỗ trợ hỏi đáp dữ liệu bằng ngôn ngữ tự nhiên (chi tiết ở Mục 12).

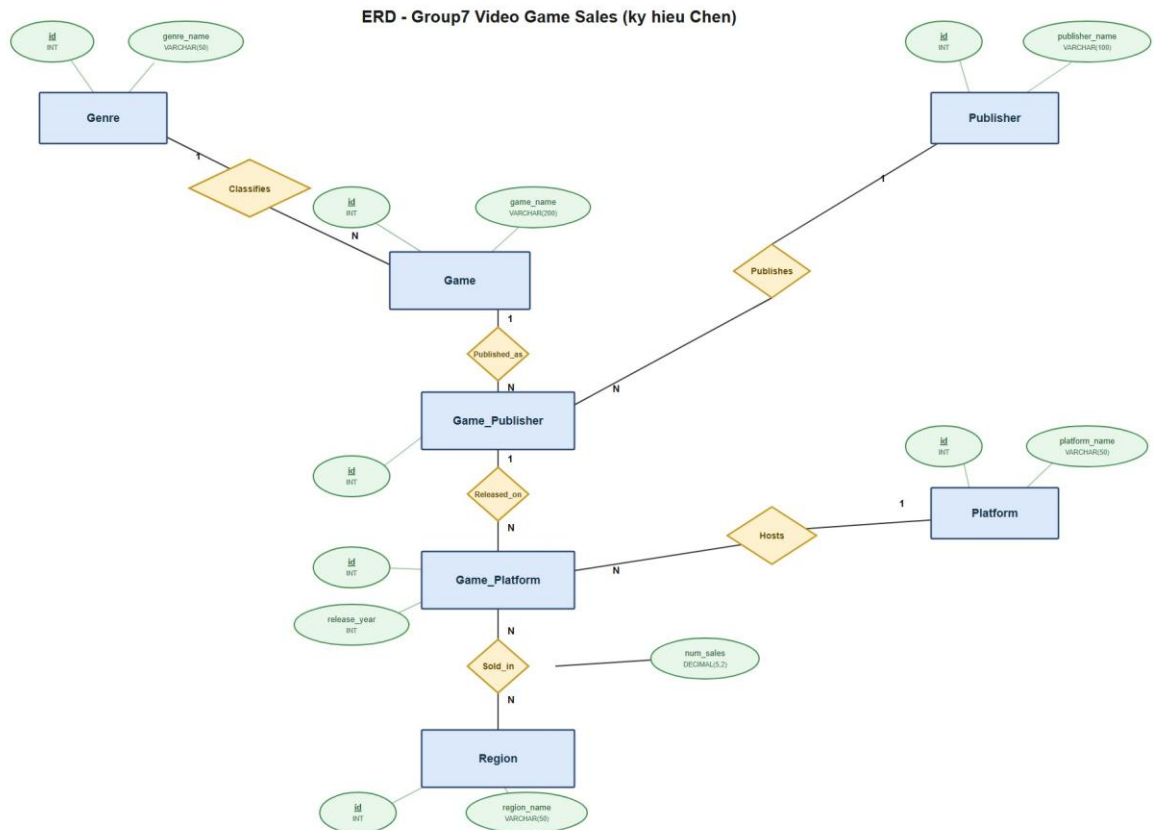
2. Mô tả nghiệp vụ

Hệ thống hỗ trợ các nghiệp vụ cụ thể sau:

- Thêm một game mới vào hệ thống, gán thể loại (genre) cho game.
- Gán game cho một nhà phát hành (publisher) - một game có thể có nhiều bản phát hành bởi các publisher khác nhau (quan hệ n-n giữa game và publisher).
- Gán một bản phát hành (game-publisher) cho một nền tảng (platform) cụ thể, kèm năm phát hành - một bản phát hành có thể lên nhiều platform (quan hệ n-n giữa game_publisher và platform).
- Ghi nhận doanh số bán (region_sales) của một bản phát hành-nền tảng tại từng khu vực (region) - quan hệ n-n giữa game_platform và region.
- Thống kê, tra cứu doanh số theo thể loại / nền tảng / nhà phát hành / khu vực.
- Hỏi đáp dữ liệu doanh số bằng ngôn ngữ tự nhiên (vd "top 5 game bán chạy nhất ở Nhật năm 2016") thay vì phải tự viết SQL - phần mở rộng ứng dụng AI (Mục 12).

3. Mô hình ER (Entity - Relationship)

Sơ đồ ER dưới đây được vẽ theo ký hiệu Chen (entity = hình chữ nhật, relationship = hình thoi, attribute = hình oval, khoá chính gạch chân), mô tả các entity game, genre, publisher, platform, region và 2 entity trung gian game_publisher, game_platform.



Hình 1. Sơ đồ ER hệ thống Group7 - Video Game Sales

Giải thích các entity và quan hệ

Bảng 1. Các thực thể và quan hệ trong mô hình ER

Entity	Vai trò	Quan hệ
Genre	Thể loại game (Action, RPG, Sports...)	1 Genre - N Game
Game	Một tựa game cụ thể	N Game - 1 Genre; 1 Game - N Game_Publisher
Publisher	Nhà phát hành game	1 Publisher - N Game_Publisher
Game_Publisher	Bảng trung gian: 1 Game do 1 Publisher phát hành	N-1 với Game, N-1 với Publisher; 1 Game_Publisher - N Game_Platform
Platform	Nền tảng chơi game (PS4, Xbox, PC...)	1 Platform - N Game_Platform
Game_Platform	Bảng trung gian: 1 bản phát hành trên 1 Platform, kèm năm phát	N-1 với Game_Publisher, N-1 với Platform; N-N với

	hành	Region
Region	Khu vực địa lý (NA, EU, JP, Other...)	N-N với Game_Platform
Sold_in	Quan hệ N-N giữa Game_Platform và Region, mang thuộc tính num_sales	N Game_Platform - N Region

Lý do thiết kế 2 bảng trung gian

- Một game có thể được nhiều publisher phát hành (n-n giữa Game và Publisher) -> tách thành thực thể trung gian Game_Publisher.
- Một bản phát hành (Game_Publisher) có thể ra mắt trên nhiều platform vào các năm khác nhau (n-n giữa Game_Publisher và Platform) -> tách thành thực thể trung gian Game_Platform.
- Doanh số gắn với từng cặp (Game_Platform, Region) cụ thể qua quan hệ N-N Sold_in (mang thuộc tính num_sales), vì cùng 1 game trên các platform/publisher khác nhau có doanh số khác nhau ở từng khu vực.

4. Mô hình quan hệ, phụ thuộc hàm và chuẩn hoá 3NF

Sau khi chuyển mô hình ER sang mô hình quan hệ (relational model), mỗi entity/relationship mang thuộc tính trở thành 1 bảng với khoá chính (PK) và khoá ngoại (FK) tương ứng, đúng như đã cài đặt trong sql/quantl3/G7_Dbscript.sql:

Bảng 2. Lược đồ quan hệ và phụ thuộc hàm của 8 bảng

Bảng	Lược đồ quan hệ (khoá chính gạch chân)	Phụ thuộc hàm
platform	platform(<u>id</u> , platform_name)	id -> platform_name
genre	genre(<u>id</u> , genre_name)	id -> genre_name
publisher	publisher(<u>id</u> , publisher_name)	id -> publisher_name
region	region(<u>id</u> , region_name)	id -> region_name
game	game(<u>id</u> , <u>genre_id</u> , game_name)	id -> genre_id, game_name
game_publisher	game_publisher(<u>id</u> , game_id, publisher_id)	id -> game_id, publisher_id
game_platform	game_platform(<u>id</u> , game_publisher_id, platform_id, release_year)	id -> game_publisher_id, platform_id, release_year

	release_year)	
region_sales	region_sales(region_id, game_platform_id, num_sales)	(region_id, game_platform_id) -> num_sales

Vì sao các bảng đạt chuẩn 3NF

Một lược đồ đạt 3NF khi đã đạt 2NF (không có phụ thuộc bộ phận vào khoá chính) và không tồn tại phụ thuộc bắc cầu (transitive dependency) - tức không có thuộc tính không khoá nào phụ thuộc vào một thuộc tính không khoá khác.

- Đạt 1NF: mọi thuộc tính đều mang giá trị nguyên tố (atomic) - không có thuộc tính đa trị hay lặp nhóm.
- Đạt 2NF: 7/8 bảng có khoá chính là 1 cột (id) nên không thể có phụ thuộc bộ phận. Riêng region_sales có khoá chính phức hợp (region_id, game_platform_id), nhưng num_sales phụ thuộc đầy đủ vào cả 2 cột của khoá, không phụ thuộc riêng vào từng cột -> không có phụ thuộc bộ phận.
- Đạt 3NF: ở từng bảng, các thuộc tính không khoá chỉ phụ thuộc trực tiếp vào khoá chính, không có thuộc tính không khoá nào xác định một thuộc tính không khoá khác (vd trong game, genre_id không xác định game_name; trong game_platform, game_publisher_id/platform_id/release_year độc lập với nhau).

Kết luận: toàn bộ 8 bảng của hệ thống Group7 đều đạt 3NF, không cần tách thêm.

5. Đặc tả yêu cầu dữ liệu (Data Dictionary)

Mỗi thuộc tính được mô tả theo mẫu: Data Element - Description - Composition or Data Type - Length - Values.

Bảng danh mục (genre, platform, publisher, region)

Bảng 3. Đặc tả các bảng danh mục (genre, platform, publisher, region)

Data Element	Description	Composition/Data Type	Length	Values
genre.id	Mã định danh duy nhất cho một thể loại game	int (Primary Key)	-	> 0, tự tăng
genre.genre_name	Tên thể loại game	varchar	50	vd: Action, Sports,

				Role-Playing
platform.id	Mã định danh duy nhất cho một nền tảng chơi game	int (Primary Key)	-	> 0, tự tăng
platform.platform_name	Tên nền tảng (console/PC)	varchar	50	vd: PS4, XOne, PC, Wii
publisher.id	Mã định danh duy nhất cho một nhà phát hành	int (Primary Key)	-	> 0, tự tăng
publisher.publisher_name	Tên nhà phát hành game	varchar	100	không rỗng
region.id	Mã định danh duy nhất cho một khu vực địa lý	int (Primary Key)	-	> 0, tự tăng
region.region_name	Tên khu vực ghi nhận doanh số	varchar	50	vd: North America, Europe, Japan, Other

Bảng game

Bảng 4. Đặc tả bảng game

Data Element	Description	Composition/Data Type	Length	Values
game.id	Mã định danh duy nhất cho một tựa game	int (Primary Key)	-	> 0, tự tăng
game.genre_id	Thể loại của game này	int (Foreign Key -> genre.id)	-	phải tồn tại trong bảng genre
game.game_name	Tên tựa game	varchar	200	không rỗng

Bảng game_publisher (bảng trung gian)

Bảng 5. Đặc tả bảng game_publisher

Data Element	Description	Composition/Data Type	Length	Values
game_publisher.id	Mã định danh duy nhất cho một bản ghi phát hành	int (Primary Key)	-	> 0, tự tăng
game_publisher.game_id	Game được phát hành	int (Foreign Key -> game.id)	-	phải tồn tại trong bảng game
game_publisher.publisher_id	Nhà phát hành thực hiện phát hành	int (Foreign Key -> publisher.id)	-	phải tồn tại trong bảng publisher

Bảng game_platform (bảng trung gian)

Bảng 6. Đặc tả bảng game_platform

Data Element	Description	Composition/Data Type	Length	Values
game_platform.id	Mã định danh duy nhất cho một bản phát hành theo nền tảng	int (Primary Key)	-	> 0, tự tăng
game_platform.game_publisher_id	Bản ghi phát hành (game + publisher) tương ứng	int (Foreign Key -> game_publisher.id)	-	phải tồn tại trong bảng game_publisher
game_platform.platform_id	Nền tảng	int (Foreign Key ->	-	phải tồn tại trong bảng

	phát hành	platform.id)		platform
game_platform.release_year	Năm phát hành trên nền tảng này	int	-	1970-2100 (ràng buộc ck_release_year)

Bảng region_sales (PK phức hợp)

Bảng 7. Đặc tả bảng region_sales

Data Element	Description	Composition/Data Type	Length	Values
region_sales.region_id	Khu vực ghi nhận doanh số	int (PK, FK - > region.id)	-	phải tồn tại trong bảng region
region_sales.game_platform_id	Bản phát hành theo nền tảng được ghi nhận doanh số	int (PK, FK - > game_platform.id)	-	phải tồn tại trong bảng game_platform
region_sales.num_sales	Doanh số bán (triệu bản) của game_platform này tại region này	decimal	(5,2)	>= 0 (ràng buộc ck_region_sales_non negative)

6. Danh sách ràng buộc dữ liệu

Trong hệ thống **Video Game Sales Management**, dữ liệu được lưu trữ trên nhiều bảng có mối quan hệ với nhau. Nếu dữ liệu không được kiểm soát, hệ thống có thể xuất hiện các thông tin không hợp lệ như tên Platform hoặc Genre bị bỏ trống, số lượng game bán ra có giá trị âm hoặc năm phát hành không hợp lý.

Để đảm bảo **tính toàn vẹn và nhất quán của dữ liệu (Data Integrity)**, nhóm xây dựng các CHECK CONSTRAINT trực tiếp trên Database.

Việc đặt Constraint ở tầng Database giúp đảm bảo các quy tắc dữ liệu được thực thi thống nhất, ngay cả khi dữ liệu được thêm hoặc cập nhật từ các nguồn khác nhau.

Toàn bộ câu lệnh SQL và ảnh kết quả thực thi: xem [Phụ lục 6](#) ở cuối báo cáo.

7. Cài đặt vật lý trên SQL Server

Việc cài đặt vật lý được thực hiện qua 2 script trong sql/trung/: 01_createDB.sql (tạo database, các bảng, khoá chính/khoá ngoại và index) và 03_insert.sql (nạp dữ liệu mẫu từ dataset Video Game Sales theo đúng thứ tự phụ thuộc khoá ngoại).

Toàn bộ câu lệnh SQL và ảnh kết quả thực thi: xem [Phụ lục 7](#) ở cuối báo cáo.

7.1. Tạo database và các bảng

Database Group7 được DROP (nếu đã tồn tại) rồi CREATE mới. Các bảng được tạo theo đúng chiều phụ thuộc khoá ngoại để không phát sinh lỗi khi CREATE TABLE: 4 bảng lookup không phụ thuộc bảng khác (platform, genre, publisher, region) được tạo trước; sau đó đến game (phụ thuộc genre); tiếp theo là 2 bảng trung gian game_publisher (phụ thuộc game, publisher) và game_platform (phụ thuộc game_publisher, platform); cuối cùng là region_sales (phụ thuộc region, game_platform).

7.2. Index phục vụ hiệu năng truy vấn

Ngoài các index tự động sinh ra cho khoá chính, script tạo thêm 8 non-clustered index trên các cột khoá ngoại thường dùng trong JOIN/WHERE của các câu truy vấn ở Mục 8, cùng 2 index hỗ trợ tìm kiếm theo tên game và lọc theo năm phát hành cho chức năng web demo.

7.3. Nạp dữ liệu mẫu (insert.sql)

Dữ liệu mẫu được tách từ dataset Video Game Sales và nạp theo đúng thứ tự phụ thuộc khoá ngoại: platform, genre, publisher, region (không phụ thuộc bảng nào) -> game (phụ thuộc genre) -> game_publisher (phụ thuộc game, publisher) -> game_platform (phụ thuộc game_publisher, platform) -> region_sales (phụ thuộc region, game_platform). Số dòng dữ liệu nạp cho từng bảng:

Bảng 8. Số dòng dữ liệu nạp cho từng bảng

Bảng	Số dòng dữ liệu
platform	31
genre	12
publisher	577
region	4
game	11.360
game_publisher	11.732
game_platform	16.326
region_sales	65.320

Dưới đây là 5 dòng dữ liệu đầu tiên minh hoạ cho từng bảng (script đầy đủ nằm trong sql/trung/03_insert.sql):

Bảng platform (31 dòng)

Bảng genre (12 dòng)

Bảng publisher (577 dòng)

Bảng region (4 dòng)

Bảng game (11.360 dòng)

Bảng game_publisher (11.732 dòng)

Bảng game_platform (16.326 dòng)

Bảng region_sales (65.320 dòng)

8. Truy vấn (Queries)

Các truy vấn dưới đây được viết trong sql/hoang/04_queries.sql (phụ trách: Hoàng). 5 câu đầu đáp ứng đủ yêu cầu bắt buộc: inner join, outer join, subquery

trong WHERE, subquery trong FROM, group by + hàm aggregate. 4 câu sau (8.6-8.9) là query insight bổ sung, phục vụ luôn cho few-shot examples của NL2SQL Agent (xem `ai/nl2sql/schema.py`).

Toàn bộ câu lệnh SQL và ảnh kết quả thực thi: xem [Phụ lục 8](#) ở cuối báo cáo.

8.1. Inner Join

Liệt kê tên game, thể loại, nhà phát hành.

8.2. Outer Join

Liệt kê tất cả game, kể cả game chưa có bản ghi doanh số.

8.3. Subquery trong WHERE

Game có bản ghi doanh số cao hơn trung bình.

8.4. Subquery trong FROM

Tổng doanh số theo thể loại.

8.5. Group By + Aggregate

Số lượng game và tổng doanh số theo nhà phát hành.

8.6. Xu hướng doanh số theo năm

Tổng doanh số toàn hệ thống theo từng năm phát hành.

8.7. Nền tảng bán chạy nhất

Tổng doanh số theo từng nền tảng (platform).

8.8. Game bán chạy nhất trên mỗi nền tảng

Dùng window function ROW_NUMBER() để lấy game có doanh số cao nhất trên từng nền tảng.

8.9. Doanh số thể loại theo khu vực

Doanh số từng thể loại (genre) tại từng khu vực (region), so sánh 2 chiều.

9. Transaction

Transaction là một nhóm các câu lệnh SQL được xem như một đơn vị xử lý duy nhất. Tất cả các thao tác thành công → COMMIT, hoặc chỉ cần một thao tác lỗi → ROLLBACK toàn bộ. Vì vậy dùng Transaction để đảm bảo tính toàn vẹn và nhất quán của dữ liệu.

Toàn bộ câu lệnh SQL và ảnh kết quả thực thi: xem [Phụ lục 9](#) ở cuối báo cáo.

10. Trigger

Trigger là một đối tượng trong database được tự động kích hoạt khi một sự kiện xảy ra. Trigger tạo ra cơ chế audit ở tầng database. Khi dữ liệu quan trọng như doanh số bị INSERT, UPDATE hoặc DELETE, hệ thống tự động ghi nhận thay đổi. Điều này giúp tăng khả năng truy vết và phát hiện các thay đổi dữ liệu bất thường.

Toàn bộ câu lệnh SQL và ảnh kết quả thực thi: xem [Phụ lục 10](#) ở cuối báo cáo.

11. Procedure / Function

Toàn bộ logic nghiệp vụ được đóng gói trong sql/trung/07_procedure.sql, gồm 5 stored procedure và 1 scalar function, giải quyết các nghiệp vụ đã mô tả ở Mục 2.

Toàn bộ câu lệnh SQL và ảnh kết quả thực thi: xem [Phụ lục 11](#) ở cuối báo cáo.

11.1. sp_GetTopGamesByRegion

Trả về Top N game bán chạy nhất tại một khu vực (region), kèm thông tin thể loại, nhà phát hành và nền tảng, sắp xếp giảm dần theo doanh số.

11.2. sp_AddNewGame

Thêm một game mới kèm thông tin publisher, platform và doanh số (nếu có) trong một transaction duy nhất: nếu bất kỳ bước insert nào lỗi, toàn bộ giao dịch được ROLLBACK để đảm bảo dữ liệu không bị ghi một phần (partial write). ID mới cho từng bảng được sinh bằng MAX(id) + 1.

11.3. sp_UpdateGameSales

Cập nhật doanh số của một game_platform tại một region cụ thể; nếu bản ghi chưa tồn tại thì insert mới (kiểu upsert). Doanh số âm bị chặn ngay từ đầu bằng RAISERROR.

11.4. sp_DeleteGame

Xoá một game cùng toàn bộ dữ liệu liên quan, theo chiều ngược của các ràng buộc khoá ngoại (region_sales -> game_platform -> game_publisher -> game) để không vi phạm FK, trong một transaction có rollback khi lỗi.

11.5. sp_SearchGames

Tìm kiếm game theo tên (LIKE), có thể lọc thêm theo genre và/hoặc platform, trả về kèm tổng doanh số - phục vụ chức năng tìm kiếm của web demo.

11.6. fn_GetTotalSalesByGame

Scalar function trả về tổng doanh số toàn cầu (tất cả platform, tất cả region) của một game, dùng để tái sử dụng logic tính tổng trong các truy vấn/báo cáo khác.

12. Áp dụng AI

Nhóm áp dụng AI theo hướng xây dựng NL2SQL Agent - chatbot nội bộ cho phép hỏi đáp dữ liệu doanh số game bằng ngôn ngữ tự nhiên, kết nối trực tiếp vào database Group7 qua API LLM ngoài (Claude):

NL2SQL Agent - hỏi đáp dữ liệu bằng ngôn ngữ tự nhiên

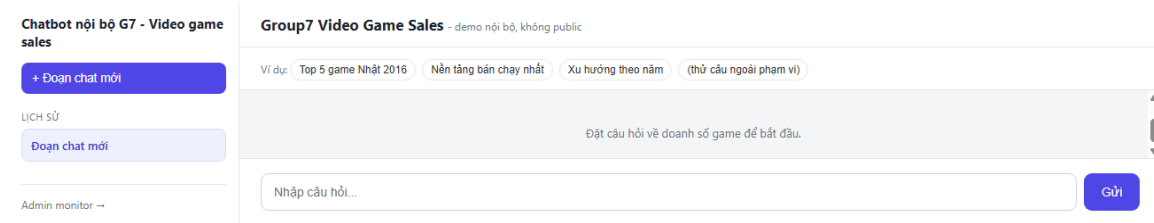
Kiến trúc pipeline: Guardrail input (chặn injection) → LLM sinh SQL → SQL Validator (parse AST, whitelist) → DB đọc read-only → LLM diễn giải kết quả → Guardrail output (grounding check) → trả lời + ghi log.

Guardrail & bảo mật:

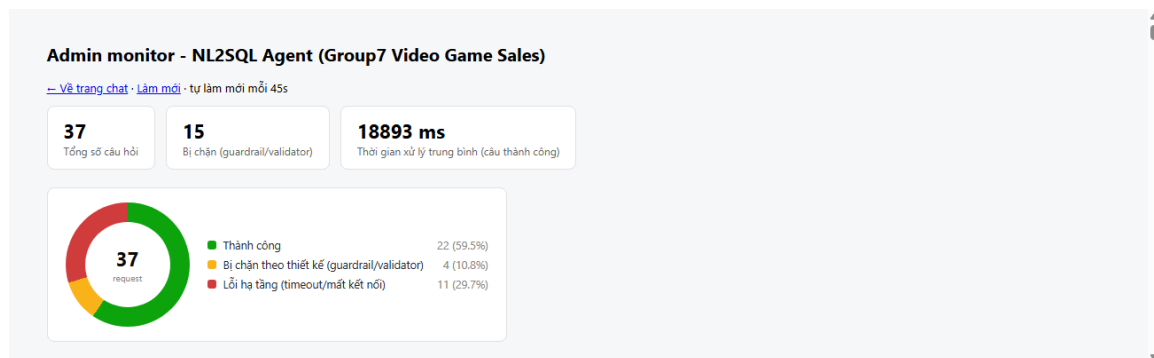
- Input: chặn prompt injection bằng regex (bắt được 1 bug thật khi test: quy tắc ban đầu bỏ sót câu "ignore all previous instructions"); lọc domain (câu hỏi có thuộc chủ đề doanh số game không) do chính LLM đánh giá, trả về token NOT_APPLICABLE nếu ngoài phạm vi - thay vì keyword-matching tĩnh dễ chặn nhầm.
- SQL Validator: parse SQL thành AST (thư viện sqlglot) thay vì regex - chỉ cho phép 1 câu SELECT, chặn cứng mọi INSERT/UPDATE/DELETE/DROP/ALTER, whitelist đúng 1 view vw_game_sales_full, tự động thêm giới hạn số dòng trả về.
- Output: grounding check - mọi số liệu LLM nêu trong câu trả lời phải khớp với kết quả SQL thật, chặn được hiện tượng LLM tự bịa số liệu (hallucination).
- Defense-in-depth ở tầng DB: mọi câu SQL đã qua validator còn được chạy dưới quyền user nl2sql_readonly (EXECUTE AS, chỉ GRANT SELECT trên view) - nếu lớp trên có lỗ hổng bypass, DB vẫn tự chặn vì user không có quyền đọc bảng gốc.

Giám sát (Monitoring): trang /admin (bảo vệ bằng HTTP Basic Auth) ghi log toàn bộ request (câu hỏi, SQL sinh ra, có bị chặn không, thời gian xử lý từng bước),

biểu đồ tròn tỷ lệ Thành công/Bị chặn/Lỗi hạ tầng, và tính năng người dùng đánh giá câu trả lời (👍 / 👎). Nhờ đo latency từng bước, phát hiện và sửa được 2 bug thật khi demo thử: (1) mở connection SQL Server mới cho mỗi câu hỏi tốn 18-118 giây do bắt tay với named instance không ổn định - sửa bằng tái sử dụng 1 connection cho cả tiến trình; (2) database hoàn toàn chưa có index ngoài khoá chính khiến 1 câu hỏi mất 8.6 giây - sau khi thêm index giảm còn 67ms (nhanh hơn ~130 lần).



Hình 2. Giao diện chat NL2SQL Agent



Hình 3. Trang /admin - giám sát request theo thời gian thực

Kiến trúc đầy đủ (bao gồm quy trình cập nhật dữ liệu, kiểm soát data poisoning, hướng phát triển production): xem [ai/NL2SQL_ARCHITECTURE.md](#).

13. Kết luận, hướng phát triển

Kết luận

Nhóm đã xây dựng hoàn chỉnh cơ sở dữ liệu quan hệ Group7 cho bài toán quản lý doanh số game: mô hình ER theo ký hiệu Chen, lược đồ quan hệ chuẩn hoá 3NF gồm 8 bảng, đặc tả dữ liệu chi tiết, và cài đặt vật lý trên SQL Server với đầy đủ ràng buộc, chỉ mục, transaction, trigger, stored procedure và function. Hệ thống được nạp 105.362 bản ghi thật từ dataset Video Game Sales và kiểm chứng bằng 9 truy vấn phân tích. Phần mở rộng NL2SQL Agent cho thấy cơ sở dữ liệu quan hệ được thiết kế tốt có thể phục vụ trực tiếp cho ứng dụng AI, với ba lớp kiểm soát

(guardrail đầu vào, kiểm tra SQL bằng AST, kiểm chứng số liệu đầu ra) và phân quyền chỉ đọc ở tầng database.

Hạn chế

- Dữ liệu doanh số hiện là dữ liệu tĩnh (dataset Video Game Sales), chưa cập nhật theo thời gian thực.
- Chưa có cơ chế phân quyền người dùng (role-based access) đầy đủ cho web demo.

Hướng phát triển

- Bổ sung bảng đánh giá/review game (rating, review) để làm giàu dữ liệu cho AI.
- Hoàn thiện NL2SQL Agent theo kiến trúc ở Mục 12, triển khai giám sát production (monitoring, guardrail) như mô tả trong ai/NL2SQL_ARCHITECTURE.md.

14. Tỷ lệ đóng góp

Bảng 9. Tỷ lệ đóng góp của các thành viên

Họ và tên	MSSV	Vai trò	Công việc phụ trách	Tỷ lệ đóng góp
Lê Minh Hoàng	HE191422	Nhóm trưởng (AI)	ER/quan hệ/3NF, data dictionary, queries, ứng dụng AI (NL2SQL), web demo, tổng hợp báo cáo, slide	45%
Phạm Thành Trung	HE170431	Thành viên (SE)	createDB, insert, procedure, web demo, báo cáo, slide	32%
Bùi Hà Vi	HE141378	Thành viên (IS)	constraints, transaction, trigger, báo cáo, slide	23%

PHỤ LỤC: CÂU LỆNH SQL VÀ KẾT QUẢ THỰC THI

Phụ lục trình bày trên khổ A4 nằm ngang, mỗi ô gồm câu lệnh SQL và ảnh chụp kết quả thực thi tương ứng. Ảnh đã được cắt về đúng vùng kết quả (Results / Messages) và giữ nguyên độ phân giải gốc nên phóng to trong Word vẫn đọc rõ.

Phụ lục 6. Ràng buộc dữ liệu

6.1. Tên Platform không được để trống

```
ALTER TABLE dbo.platform
ADD CONSTRAINT CK_platform_name_not_empty
CHECK (LEN(LTRIM(RTRIM(platform_name))) > 0);
```

Minh hoạ bằng câu lệnh kiểm tra ở trên.

6.2. Tên Genre không được để trống

```
ALTER TABLE dbo.genre
ADD CONSTRAINT CK_genre_name_not_empty
CHECK (LEN(LTRIM(RTRIM(genre_name))) > 0);
```

Minh hoạ bằng câu lệnh kiểm tra ở trên.

6.3. Doanh số không được âm

```
ALTER TABLE dbo.region_sales
ADD CONSTRAINT CK_region_sales_non_negative
CHECK (num_sales >= 0);
```

Minh hoạ bằng câu lệnh kiểm tra ở trên.

6.4. Năm phát hành hợp lệ

```
ALTER TABLE dbo.game_platform
ADD CONSTRAINT CK_game_platform_release_year
CHECK (
    release_year IS NULL
    OR release_year BETWEEN 1980 AND 2026
);
```

Minh hoạ bằng câu lệnh kiểm tra ở trên.

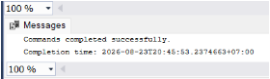
Phụ lục 7. Cài đặt vật lý trên SQL Server

7.1. Tạo database và các bảng

```
USE [master]
GO
CREATE DATABASE [Group7]
GO
USE [Group7]
GO

-- Bang chinh (co FK don)
CREATE TABLE game (
    id INT PRIMARY KEY,
    genre_id INT NOT NULL,
    game_name VARCHAR(200) NOT NULL,
    CONSTRAINT fk_game_genre FOREIGN KEY (genre_id) REFERENCES genre(id)
);

-- Bang co khoa chinh phuc hop (2 bang trung gian con lai: game_publisher,
-- game_platform cau truc tuong tu, chi khac cot)
CREATE TABLE region_sales (
    region_id INT NOT NULL,
    game_platform_id INT NOT NULL,
    num_sales DECIMAL(5, 2),
    CONSTRAINT pk_region_sales PRIMARY KEY (region_id, game_platform_id),
    CONSTRAINT fk_rs_region FOREIGN KEY (region_id) REFERENCES region(id),
    CONSTRAINT fk_rs_gameplatform FOREIGN KEY (game_platform_id) REFERENCES game_platform(id)
);
-- (trich luoc - xem day du trong 01_createDB.sql)
```



7.3.1. Nạp bảng platform (31 dòng)

```
INSERT INTO platfrom VALUES
(1,'Wii'),
(2,'NES'),
(3,'GB'),
(4,'DS'),
(5,'X360'),
... -- tổng cộng 31 dòng
```



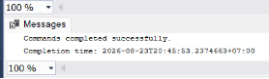
7.3.3. Nạp bảng publisher (577 dòng)

```
INSERT INTO publisher VALUES
(1,'10TACLE Studios'),
(2,'1C Company'),
(3,'20th Century Fox Video Games'),
(4,'2D Boy'),
(5,'3DO'),
... -- tổng cộng 577 dòng
```



7.2. Index phục vụ truy vấn

```
CREATE NONCLUSTERED INDEX ix_game_genre_id ON game (genre_id);
CREATE NONCLUSTERED INDEX ix_game_publisher_game_id ON game_publisher (game_id);
CREATE NONCLUSTERED INDEX ix_game_publisher_publisher_id ON game_publisher (publisher_id);
CREATE NONCLUSTERED INDEX ix_game_platform_game_publisher_id ON game_platform (game_publisher_id);
CREATE NONCLUSTERED INDEX ix_game_platform_platform_id ON game_platform (platform_id);
CREATE NONCLUSTERED INDEX ix_region_sales_game_platform_id ON region_sales (game_platform_id);
CREATE NONCLUSTERED INDEX ix_game_name ON game (game_name);
CREATE NONCLUSTERED INDEX ix_game_platform_release_year ON game_platform (release_year);
```



7.3.2. Nạp bảng genre (12 dòng)

```
INSERT INTO genre VALUES
(1,'Action'),
(2,'Adventure'),
(3,'Fighting'),
(4,'Misc'),
(5,'Platform'),
... -- tổng cộng 12 dòng
```



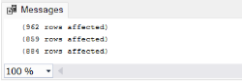
7.3.4. Nạp bảng region (4 dòng)

```
INSERT INTO region VALUES
(1,'North America'),
(2,'Europe'),
(3,'Japan'),
(4,'Other');
```



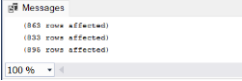
7.3.5. Nạp bảng game (11.360 dòng)

```
INSERT INTO game (id, genre_id, game_name) VALUES
(1,11, '''98 Koshien'),
(2,8, '.hack//G.U. Vol.1//Rebirth'),
(3,8, '.hack//G.U. Vol.2//Reminisce'),
(4,8, '.hack//G.U. Vol.2//Reminisce (jp sales)'),
(5,8, '.hack//G.U. Vol.3//Redemption'),
... -- tổng cộng 11.360 dòng
```



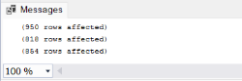
7.3.7. Nạp bảng game_platform (16.326 dòng)

```
INSERT INTO game_platform (id, game_publisher_id, platform_id, release_year) VALUES
(1,8564,4,2007),
(2,9852,4,2007),
(3,11063,7,2006),
(4,9065,15,2011),
(5,9544,15,2011),
... -- tổng cộng 16.326 dòng
```



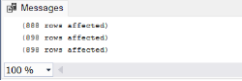
7.3.6. Nạp bảng game_publisher (11.732 dòng)

```
INSERT INTO game_publisher (id, game_id, publisher_id) VALUES
(1,10866,369),
(2,9244,369),
(3,5464,369),
(4,10868,369),
(5,3462,369),
... -- tổng cộng 11.732 dòng
```



7.3.8. Nạp bảng region_sales (65.320 dòng)

```
INSERT INTO region_sales (region_id, game_platform_id, num_sales) VALUES
(1,50,3.50),
(1,51,1.43),
(1,52,0.51),
(1,53,0.27),
(1,54,0.48),
... -- tổng cộng 65.320 dòng
```



Phụ lục 8. Truy vấn

8.1. Inner Join

```
SELECT g.game_name, gr.genre_name, p.publisher_name
FROM game g
INNER JOIN genre gr ON g.genre_id = gr.id
INNER JOIN game_publisher gp ON gp.game_id = g.id
INNER JOIN publisher p ON p.id = gp.publisher_id;
```

100 % ▾ No issues found

	game_name	genre_name	publisher_name
1	'98 Koshien	Sports	Magical Company
2	hack/IG.U. Vol.1/Rebirth	Role-Playing	Namco Bandai Games
3	hack/IG.U. Vol.2/Reminisc	Role-Playing	Namco Bandai Games
4	hack/IG.U. Vol.2/Reminisc (jp sales)	Role-Playing	Namco Bandai Games
5	hack/IG.U. Vol.3/Redemption	Role-Playing	Namco Bandai Games
6	hack/Infection Part 1	Role-Playing	Atari
7	hack/Luk	Role-Playing	Namco Bandai Games
8	hack/Mutation Part 2	Role-Playing	Atari
9	hack/Outbreak Part 3	Role-Playing	Atari
10	hack/Quarantine Part 4: The Final Chapter	Role-Playing	Atari
11	hack: Sekai no Mukou ni + Versus	Action	Namco Bandai Games
12	007 Racing	Racing	Electronic Arts
13	007 Quantum of Solace	Action	Activision

8.3. Subquery trong WHERE

```
SELECT DISTINCT g.game_name
FROM game g
JOIN game_publisher gp ON gp.game_id = g.id
JOIN game_platform gpl ON gpl.game_publisher_id = gp.id
JOIN region_sales rs ON rs.game_platform_id = gpl.id
WHERE rs.num_sales > (SELECT AVG(num_sales) FROM region_sales);
```

100 % ▾ No issues found

	game_name
1	Jikkyou Powerful Pro Yakyuu 6
2	The Lord of the Rings: The Fellowship of the Ring
3	The Sims: Hot Date
4	Atari Anthology
5	Chocobo Racing
6	Dark Cloud
7	Disney's Chicken Little
8	Dragon Quest III: Goshite Densetsu e...
9	The Elder Scrolls III: Morrowind
10	Animal Paradise
11	Ape Escape: On the Loose

8.5. Group By + Aggregate

```
SELECT p.publisher_name, COUNT(DISTINCT g.id) AS so_luong_game,
SUM(rs.num_sales) AS tong_doanh_so
FROM publisher p
JOIN game_publisher gp ON gp.publisher_id = p.id
JOIN game g ON g.id = gp.game_id
JOIN game_platform gpl ON gpl.game_publisher_id = gp.id
JOIN region_sales rs ON rs.game_platform_id = gpl.id
GROUP BY p.publisher_name
ORDER BY tong_doanh_so DESC;
```

100 % ▾ No issues found

	publisher_name	so_luong_game	tong_doanh_so
1	Nintendo	662	1784.23
2	Electronic Arts	605	1096.45
3	Activision	414	721.06
4	Sony Computer Entertainment	621	607.27
5	Ubisoft	571	473.33
6	Take-Two Interactive	201	399.45

8.2. Outer Join

```
SELECT g.game_name, rs.num_sales
FROM game g
LEFT JOIN game_publisher gp ON gp.game_id = g.id
LEFT JOIN game_platform gpl ON gpl.game_publisher_id = gp.id
LEFT JOIN region_sales rs ON rs.game_platform_id = gpl.id;
```

100 % ▾ No issues found

	game_name	num_sales
1	'98 Koshien	0.15
2	'98 Koshien	0.10
3	'98 Koshien	0.12
4	'98 Koshien	0.03
5	hack/IG.U. Vol.1/Rebirth	0.00
6	hack/IG.U. Vol.1/Rebirth	0.00
7	hack/IG.U. Vol.1/Rebirth	0.17
8	hack/IG.U. Vol.1/Rebirth	0.00
9	hack/IG.U. Vol.2/Reminisc	0.11
10	hack/IG.U. Vol.2/Reminisc	0.09

8.4. Subquery trong FROM

```
SELECT genre_name, total_sales
FROM (
    SELECT gr.genre_name, SUM(rs.num_sales) AS total_sales
    FROM genre gr
    JOIN game g ON g.genre_id = gr.id
    JOIN game_publisher gp ON gp.game_id = g.id
    JOIN game_platform gpl ON gpl.game_publisher_id = gp.id
    JOIN region_sales rs ON rs.game_platform_id = gpl.id
    GROUP BY gr.genre_name
) AS genre_sales
ORDER BY total_sales DESC;
```

100 % ▾ No issues found

	genre_name	total_sales
1	Action	1722.05
2	Sports	1308.83
3	Shooter	1025.75
4	Role-Playing	923.75
5	Platform	828.82
6	Misc	796.83

8.6. Xu hướng doanh số theo năm

```
SELECT gpl.release_year, SUM(rs.num_sales) AS tong_doanh_so
FROM game_platform gpl
JOIN region_sales rs ON rs.game_platform_id = gpl.id
GROUP BY gpl.release_year
ORDER BY gpl.release_year;
```

100 % ▾ No issues found

	release_year	tong_doanh_so
1	1989	11.38

8.7. Nền tảng bán chạy nhất

```
SELECT pf.platform_name, SUM(rs.num_sales) AS tong_doanh_so
FROM platform pf
JOIN game_platform gpl ON gpl.platform_id = pf.id
JOIN region_sales rs ON rs.game_platform_id = gpl.id
GROUP BY pf.platform_name
ORDER BY tong_doanh_so DESC;
```

100 %
Results Messages

	platform_name	tong_doanh_so
1	PS2	1233.56

8.9. Doanh số thể loại theo khu vực

```
SELECT gr.genre_name, r.region_name, SUM(rs.num_sales) AS tong_doanh_so
FROM genre gr
JOIN game g ON g.genre_id = gr.id
JOIN game_publisher gp ON gp.game_id = g.id
JOIN game_platform gpl ON gpl.game_publisher_id = gp.id
JOIN region_sales rs ON rs.game_platform_id = gpl.id
JOIN region r ON r.id = rs.region_id
GROUP BY gr.genre_name, r.region_name
ORDER BY gr.genre_name, tong_doanh_so DESC;
```

100 %
Results Messages

	genre_name	region_name	tong_doanh_so
1	Action	North America	861.80

8.8. Game bán chạy nhất mỗi nền tảng

```
SELECT platform_name, game_name, tong_doanh_so
FROM (
    SELECT pf.platform_name, g.game_name,
           SUM(rs.num_sales) AS tong_doanh_so,
           ROW_NUMBER() OVER (
               PARTITION BY pf.platform_name
               ORDER BY SUM(rs.num_sales) DESC) AS rn
    FROM platform pf
    JOIN game_platform gpl ON gpl.platform_id = pf.id
    JOIN game_publisher gp ON gp.id = gpl.game_publisher_id
    JOIN game g ON g.id = gp.game_id
    JOIN region_sales rs ON rs.game_platform_id = gpl.id
    GROUP BY pf.platform_name, g.game_name
) t
WHERE rn = 1;
```

100 %
Results Messages

	platform_name	game_name	tong_doanh_so
1	2600	Pac-Man	7.51

Phụ lục 9-10. Transaction và Trigger

9.1. Transaction nạp một game mới (atomic)

```
USE [Group7];
GO
/* =====
Requirement 8: Transaction
Insert one new game, publisher relation, platform relation,
and four regional sales records as one atomic transaction.
===== */
BEGIN TRY
    BEGIN TRANSACTION;
    DECLARE @GameID INT =
        (SELECT ISNULL(MAX(id), 0) + 1 FROM dbo.game);
    DECLARE @GamePublisherID INT =
        (SELECT ISNULL(MAX(id), 0) + 1 FROM dbo.game_publisher);
    DECLARE @GamePlatformID INT =
        (SELECT ISNULL(MAX(id), 0) + 1 FROM dbo.game_platform);
    /* Use existing Nintendo publisher and Wii platform */
    INSERT INTO dbo.game (id, genre_id, game_name)
    VALUES (@GameID, 1, 'DBI202 Assignment Demo Game');
    INSERT INTO dbo.game_publisher (id, game_id, publisher_id)
    VALUES (@GamePublisherID, @GameID, 369);
    INSERT INTO dbo.game_platform
        (id, game_publisher_id, platform_id, release_year)
    VALUES
        (@GamePlatformID, @GamePublisherID, 1, 2026);
    INSERT INTO dbo.region_sales
        (region_id, game_platform_id, num_sales)
    VALUES
        (1, @GamePlatformID, 1.20),
        (2, @GamePlatformID, 0.80),
        (3, @GamePlatformID, 0.50),
        (4, @GamePlatformID, 0.20);
    COMMIT TRANSACTION;
    PRINT 'Transaction committed successfully.';
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0
        ROLLBACK TRANSACTION;
    THROW;
END CATCH;
GO
```

Minh hoạ bằng câu lệnh kiểm tra ở trên.

10.1. Trigger audit region_sales (phần 1)

```
USE [Group7];
GO
/* =====
Requirement 9: Trigger
Audit INSERT / UPDATE / DELETE actions on region_sales.
===== */
IF OBJECT_ID('dbo.region_sales_audit', 'U') IS NOT NULL
    DROP TABLE dbo.region_sales_audit;
GO
CREATE TABLE dbo.region_sales_audit
(
    audit_id INT IDENTITY(1,1) PRIMARY KEY,
    action_type VARCHAR(10) NOT NULL,
    region_id INT NULL,
    game_platform_id INT NULL,
    old_num_sales DECIMAL(5,2) NULL,
    new_num_sales DECIMAL(5,2) NULL,
    changed_at DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);
GO
IF OBJECT_ID('dbo.trg_region_sales_audit', 'TR') IS NOT NULL
    DROP TRIGGER dbo.trg_region_sales_audit;
GO
CREATE TRIGGER dbo.trg_region_sales_audit
ON dbo.region_sales
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    /* INSERT */
    IF EXISTS (SELECT 1 FROM inserted)
        AND NOT EXISTS (SELECT 1 FROM deleted)
    BEGIN
        INSERT INTO dbo.region_sales_audit
            (action_type, region_id, game_platform_id,
             old_num_sales, new_num_sales)
        SELECT
            'INSERT',
            region_id,
            game_platform_id,
            NULL,
            num_sales
        FROM inserted;
    END;
END;
```

Minh hoạ bằng câu lệnh kiểm tra ở trên.

10.2. Trigger audit region_sales (phần 2)

```
/* DELETE */
IF EXISTS (SELECT 1 FROM deleted)
    AND NOT EXISTS (SELECT 1 FROM inserted)
BEGIN
    INSERT INTO dbo.region_sales_audit
        (action_type, region_id, game_platform_id,
         old_num_sales, new_num_sales)
    SELECT
        'DELETE',
        region_id,
        game_platform_id,
        num_sales,
        NULL
    FROM deleted;
END;
/* UPDATE: record old and new values separately.
This also works with the duplicate pairs existing
in the supplied dataset. */
IF EXISTS (SELECT 1 FROM inserted)
    AND EXISTS (SELECT 1 FROM deleted)
BEGIN
    INSERT INTO dbo.region_sales_audit
        (action_type, region_id, game_platform_id,
         old_num_sales, new_num_sales)
    SELECT
        'UPD_OLD',
        region_id,
        game_platform_id,
        num_sales,
        NULL
    FROM deleted;
    INSERT INTO dbo.region_sales_audit
        (action_type, region_id, game_platform_id,
         old_num_sales, new_num_sales)
    SELECT
        'UPD_NEW',
        region_id,
        game_platform_id,
        NULL,
        num_sales
    FROM inserted;
END;
END;
GO
```

Minh hoạ bằng câu lệnh kiểm tra ở trên.

10.3. Câu lệnh kiểm tra trigger

```
-- Kiểm tra trigger trg_region_sales_audit
INSERT INTO region_sales (region_id, game_platform_id, num_sales) VALUES (1, 1, 2.50);
SELECT TOP 5 * FROM region_sales_audit ORDER BY audit_id DESC;    -- 1 dòng 'INSERT'

UPDATE region_sales SET num_sales = 3.00 WHERE region_id = 1 AND game_platform_id = 1;
SELECT TOP 5 * FROM region_sales_audit ORDER BY audit_id DESC;    -- 2 dòng 'UPD_OLD' + 'UPD_NEW'

DELETE FROM region_sales WHERE region_id = 1 AND game_platform_id = 1;
SELECT TOP 5 * FROM region_sales_audit ORDER BY audit_id DESC;    -- 1 dòng 'DELETE'
```

Minh hoạ bằng câu lệnh kiểm tra ở trên.

Phụ lục 11. Procedure / Function

11.1. sp_GetTopGamesByRegion

```
CREATE OR ALTER PROCEDURE sp_GetTopGamesByRegion
    @RegionId INT,
    @TopN INT = 10
AS
BEGIN
    SELECT TOP (@TopN) g.game_name, gr.genre_name, p.publisher_name,
        pl.platform_name, rs.num_sales
    FROM region_sales rs
    JOIN game_platform gpl ON gpl.id = rs.game_platform_id
    JOIN game_publisher gp ON gp.id = gpl.game_publisher_id
    JOIN game g ON g.id = gp.game_id
    JOIN genre gr ON gr.id = g.genre_id
    JOIN publisher p ON p.id = gp.publisher_id
    JOIN platform pl ON pl.id = gpl.platform_id
    WHERE rs.region_id = @RegionId
    ORDER BY rs.num_sales DESC;
END;
-- (trich luoc - xem day du trong 07_procedure.sql)
```

```
100 % ▾
Messages
Commands completed successfully.
Completion time: 2026-08-23T20:50:04.3332189+07:00
100 % ▾
```

11.3. sp_UpdateGameSales

```
CREATE OR ALTER PROCEDURE sp_UpdateGameSales
    @GamePlatformId INT, @RegionId INT, @NewSales DECIMAL(5,2)
AS
BEGIN
    IF @NewSales < 0
    BEGIN
        RAISERROR('Doanh so khong duoc am', 16, 1);
        RETURN;
    END

    IF EXISTS (SELECT 1 FROM region_sales WHERE game_platform_id = @GamePlatformId AND region_id =
@RegionId)
        UPDATE region_sales SET num_sales = @NewSales
        WHERE game_platform_id = @GamePlatformId AND region_id = @RegionId;
    ELSE
        INSERT INTO region_sales (region_id, game_platform_id, num_sales)
        VALUES (@RegionId, @GamePlatformId, @NewSales);
END;
```

```
100 % ▾
Messages
Commands completed successfully.
Completion time: 2026-08-23T20:51:53.4638720+07:00
100 % ▾
```

11.2. sp_AddNewGame

```
CREATE OR ALTER PROCEDURE sp_AddNewGame
    @GameName VARCHAR(200), @GenreId INT, @PublisherId INT, @PlatformId INT,
    @ReleaseYear INT = NULL, @RegionId INT = NULL, @NumSales DECIMAL(5,2) = NULL,
    @NewGameId INT OUTPUT
AS
BEGIN
    BEGIN TRANSACTION;
    BEGIN TRY
        INSERT INTO game (id, genre_id, game_name) VALUES (@NewGameId, @GenreId, @GameName);
        INSERT INTO game_publisher (id, game_id, publisher_id) VALUES (... , @NewGameId, @PublisherId);
        INSERT INTO game_platform (id, game_publisher_id, platform_id, release_year) VALUES (... ,
@PlatformId, @ReleaseYear);
        -- insert them region_sales neu co truyen @RegionId/@NumSales
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
-- (trich luoc - xem day du trong 07_procedure.sql)
```

```
100 % ▾
Messages
Commands completed successfully.
Completion time: 2026-08-23T20:51:04.3470568+07:00
100 % ▾
```

11.4. sp_DeleteGame

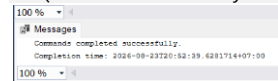
```
CREATE OR ALTER PROCEDURE sp_DeleteGame
    @GameId INT
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM game WHERE id = @GameId)
    BEGIN
        RAISERROR('Khong tim thay game_id=%d', 16, 1, @GameId);
        RETURN;
    END

    BEGIN TRANSACTION;
    BEGIN TRY
        -- Xoa theo chieu nguoc FK: region_sales -> game_platform -> game_publisher -> game
        DELETE rs FROM region_sales rs JOIN game_platform gpl ON gpl.id = rs.game_platform_id
        JOIN game_publisher gp ON gp.id = gpl.game_publisher_id WHERE gp.game_id = @GameId;
        DELETE gpl FROM game_platform gpl JOIN game_publisher gp ON gp.id = gpl.game_publisher_id
        WHERE gp.game_id = @GameId;
        DELETE FROM game_publisher WHERE game_id = @GameId;
        DELETE FROM game WHERE id = @GameId;
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

```
100 % ▾
Messages
Commands completed successfully.
Completion time: 2026-08-23T20:52:16.5469682+07:00
100 % ▾
```

11.5. sp_SearchGames

```
CREATE OR ALTER PROCEDURE sp_SearchGames
    @Keyword VARCHAR(200) = NULL, @GenreId INT = NULL,
    @PlatformId INT = NULL, @TopN INT = 50
AS
BEGIN
    SELECT TOP (@TopN) g.game_name, gr.genre_name, pl.platform_name,
        ISNULL(SUM(rs.num_sales), 0) AS total_sales
    FROM game g
    JOIN genre gr ON gr.id = g.genre_id
    LEFT JOIN game_publisher gp ON gp.game_id = g.id
    -- ... (JOIN game_platform/platform/region_sales tuong tu 11.1)
    WHERE (@Keyword IS NULL OR g.game_name LIKE '%' + @Keyword + '%')
        AND (@GenreId IS NULL OR g.genre_id = @GenreId)
        AND (@PlatformId IS NULL OR gp.platform_id = @PlatformId)
    GROUP BY g.id, g.game_name, gr.genre_name, pl.platform_name
    ORDER BY total_sales DESC;
END;
-- (trich luoc - xem day du trong 07_procedure.sql)
```



11.6. fn_GetTotalSalesByGame

```
CREATE OR ALTER FUNCTION fn_GetTotalSalesByGame(@GameId INT)
RETURNS DECIMAL(10, 2)
AS
BEGIN
    DECLARE @Total DECIMAL(10, 2);

    SELECT @Total = ISNULL(SUM(rs.num_sales), 0)
    FROM region_sales rs
    JOIN game_platform gpl ON gpl.id = rs.game_platform_id
    JOIN game_publisher gp ON gp.id = gpl.game_publisher_id
    WHERE gp.game_id = @GameId;

    RETURN @Total;
END;
```

